Aeronautics and Space Technology Club

IIT (ISM) Dhanbad

Signal Processing Ladder: A Hands-On Problem Series from Sensors to Machine Learning

Problem Statement 08 · Part C: Electronics for Signal Processing Level 2

Track: Analog Filter Theory, RC and Active Filters, Filter Families, Tolerance Analysis, Anti-Alias Design

Prerequisites: Problem 05 · **Estimated effort:** 10–14 hours

Analog Filters: Shaping the Signal Before the Converter Sees It

Resistors, capacitors and an op-amp implement the transfer functions of problem 05 in hardware. Write your own circuit solver, design a Butterworth filter from a specification, compare the four classic approximations, realise a Sallen–Key stage with real component values, test how tolerances spoil it, and size the anti-alias filter that protects an ADC.

General Instructions

- **Thorough Review:** Read the whole problem statement, including the theory, before writing any code. Do the short analytic derivations *by hand first*; the code is there to confirm and extend your understanding, not to replace it.
- **Pacing:** This is a self-paced problem. The split below is a guide for the effort estimate in the header.

Stage	Phase	What you should have at the end of it
1	Think (∼20%)	Theory understood; every analytic result derived; signals, units and sampling rates written down.
2	Build (∼30%)	Core code working end to end on the baseline data and validated against the analytic results.
3	Experiment (∼30%)	Full sweeps, stress tests and the real or synthetic datasets processed; first plots and numbers.
4	Refine (∼20%)	Weaknesses fixed, honest analysis and memo written, repository folder clean.

- **Compute and Tools:** Everything is designed to run on a laptop CPU in Python 3.10+ (NumPy, SciPy, Matplotlib, and where stated pandas, Astropy, scikit-learn, PyWavelets and PyTorch). No GPU and no hardware are required; where a problem offers an optional hardware experiment it is marked as a bonus. A full run should never need more than a few minutes; if yours does, vectorise before you scale up.
- **Data:** Each problem lists (i) a *synthetic generator* from `sp_common.synth` (offline, seeded, with known ground truth) and (ii) *real datasets* to download yourself. Develop and validate on the synthetic data first, then repeat the key result on real data. Never commit large raw data files to the repository.
- **Submission Format:** Push your work to `solutions/08_analog_filters/` in the repository: (i) clean, commented, reproducible Jupyter notebook(s) with fixed random seeds, (ii) a short written report (`report.pdf` or `report.md`) containing the requested analysis sections, and (iii)

all figures in a `figures/` sub-folder. The report must be understandable without running the code.

- **Originality:** All code, analysis and written explanations must be your own work. Any external library, snippet, paper or AI tool you use must be cited in the report. You should be able to explain every line you submit.
- **Honesty over Hype:** Validate against analytic results *before* trusting a number. A modest or negative result that is correctly analysed is worth more than an inflated claim. Report error bars, false-alarm rates and failure cases, not only your best run.
- **Units:** Write the units of every quantity and state in the first notebook cell whether a frequency is in Hz or rad/s and whether it is normalised to the sampling rate. Most mistakes in signal processing are factors of 2π, factors of 2 in one-sided spectra, or an off-by-one in the frequency axis.
- **Getting Stuck:** Re-read the theory section, check the self-check targets, then ask your mentor or open an issue in the repository. Describe what you expected, what you got, and what you already tried.

Background and Motivation

Before a sensor voltage reaches an ADC it passes through an analog filter that removes noise and interference outside the band of interest and, most importantly, anything that would alias after sampling (problem 02). This filter is built from a handful of passive parts and an operational amplifier, and its few components must meet a specification on bandwidth, stopband attenuation, ripple and delay. The art is in choosing a filter family whose trade-offs match the application: Butterworth for a flat passband, Chebyshev for the steepest roll-off, Bessel for undistorted pulse shape, elliptic for the lowest order.

In this problem you move between three levels: the transfer function, the circuit that realises it, and the real-world tolerance of its components. You will implement an AC circuit solver yourself, so the circuit analysis is not a black box.

Theoretical Framework

1. First-Order Filters

The RC low-pass is $H(s) = 1/(1+sRC)$ with corner $f_c = 1/(2\pi RC)$, magnitude $|H| = 1/\sqrt{1+(f/f_c)^2}$, phase $-\arctan(f/f_c)$ and group delay $\tau_g = -\mathrm{d}\phi/\mathrm{d}\omega = RC/(1+(\omega RC)^2)$, equal to RC at DC. Cascading two unbuffered RC stages loads the first and does not produce $1/(1+sRC)^2$; an op-amp buffer between them does.

2. Filter Families

Butterworth (maximally flat): $|H(\mathrm{j}\omega)|^2 = 1/(1+(\omega/\omega_c)^{2n})$, with the n poles equally spaced on the left half of a circle of radius ω_c at angles $\theta_k = \pi(2k+n-1)/(2n)$, $k = 1 \ldots n$. The order needed to meet a passband attenuation A_p at ω_p and a stopband attenuation A_s at ω_s is

$$n \;\geq\; \frac{\log_{10}\!\left[(10^{A_s/10}-1)/(10^{A_p/10}-1)\right]}{2\log_{10}(\omega_s/\omega_p)}. \tag{1}$$

Chebyshev type I trades passband ripple for a steeper edge; *Bessel* has maximally flat group delay (best pulse fidelity, gentlest edge); *elliptic* has ripple in both bands and the lowest order for a given specification but the worst phase.

3. Second-Order Sections and Sallen–Key

Any higher-order filter is a cascade of second-order sections $H(s) = \omega_0^2/(s^2 + (\omega_0/Q)s + \omega_0^2)$. The unity-gain Sallen–Key low-pass (resistors R_1, R_2 in series, C_2 from the op-amp input to ground, C_1 from the junction to the output) has

$$\omega_0 = \frac{1}{\sqrt{R_1 R_2 C_1 C_2}}, \qquad Q = \frac{\sqrt{R_1 R_2 C_1 C_2}}{C_2(R_1 + R_2)}. \tag{2}$$

With $R_1 = R_2 = R$ this gives $Q = \frac{1}{2}\sqrt{C_1/C_2}$, so a Butterworth section ($Q = 1/\sqrt{2}$) needs $C_1 = 2C_2$.

4. Modified Nodal Analysis

For a linear circuit at frequency ω, write Kirchhoff's current law at every node with admittances $Y_R = 1/R$ and $Y_C = \mathrm{j}\omega C$ and solve $\mathbf{Y}(\omega)\mathbf{v} = \mathbf{i}$; an ideal op-amp (or buffer) is added as a constraint $v_+ = v_-$ with an extra unknown for its output current.

5. Anti-Alias Specification

To protect a B-bit ADC sampling at f_s a signal band up to $f_{\max}$, interferers at $f_s - f_{\max}$ must be attenuated below the quantisation floor, i.e. by at least about $6.02B + 1.76$ dB. A Butterworth filter of order n attenuates by about $20n \log_{10}(f/f_c)$ dB far above its corner.

Data and Simulation Specification

RC stage	$R = 1.6\,\mathrm{k\Omega}$, $C = 100\,\mathrm{nF}$.
Butterworth spec	Passband to 1 kHz with at most 1 dB loss; stopband from 4 kHz with at least 40 dB.
Family comparison	Order 5, cutoff 1 kHz; Chebyshev I with 1 dB ripple, elliptic with 1 dB ripple and 40 dB stopband.
Sallen–Key	Target 2nd-order Butterworth at 1 kHz; $R_1 = R_2 = 10\,\mathrm{k\Omega}$; tolerance studies with 1%, 5% and 10% parts, 5000 Monte Carlo trials.
ADC	12-bit, $f_s = 10\,\mathrm{kHz}$, signal band 0–1 kHz; alternative with 4× oversampling ($f_s = 40\,\mathrm{kHz}$).
Measurement (real data)	Build the RC or Sallen–Key filter on a breadboard, drive it with a function generator (or the phone's tone generator and a sound-card line input) and measure gain and phase at 10–20 frequencies with an oscilloscope or sound-card recorder; export the table to CSV.

Your Task

1. **First-order response (Stage 2).** Derive $|H|$, phase, group delay and the step response of the RC stage. Compute f_c for the stated R and C and verify it against `scipy.signal.freqs`. Show that two unbuffered RC stages give a different response from $1/(1 + sRC)^2$, with the exact transfer function derived by hand.
2. **Your own circuit solver.** Implement modified nodal analysis in NumPy for networks of resistors, capacitors and ideal op-amps and solve the AC response over a log frequency grid. Reproduce the RC filter, the unbuffered two-stage RC and the Sallen–Key stage, and compare with the analytic $H(s)$.
3. **Butterworth from specification.** Evaluate Eq. (1) for the stated specification, round up, compute the pole positions from the formula and plot them on the pole circle. Confirm

the response with `scipy.signal.buttord` and `butter`. Show the passband, 3 dB point and stopband on one annotated plot.

4. **Comparing the families.** For order 5 compute magnitude, phase, group delay and step response of Butterworth, Chebyshev I, Bessel and elliptic filters. Tabulate the roll-off at $2f_c$, peak group-delay variation and step overshoot, and say which you would pick for (a) an accelerometer channel that will be analysed spectrally, (b) a pulse-timing channel.

5. **Sallen–Key design and tolerance.** Pick component values from Eq. (2) (round to E12 values) and verify f_0 and Q. Run the Monte Carlo tolerance study; plot the histograms of f_0 and Q and the family of magnitude responses, and report the yield for f_0 within $\pm 5\%$ and Q within $\pm 10\%$ at each tolerance.

6. **Anti-alias design.** For the 12-bit, 10 kHz ADC, compute the attenuation required at $f_s - f_{\max}$ and the Butterworth order needed with a 1.2 kHz corner. Repeat for $f_s = 40$ kHz. Compare the group delay and passband droop at 1 kHz of the two designs.

7. **Measurement.** Compare your measured gain and phase with the model, report the deviations and explain them with real component values (tolerance, op-amp bandwidth, source and load impedance).

Self-check targets

$f_c = 994.7$ Hz for 1.6 kΩ and 100 nF. The specification requires Butterworth order $n = 4$ (formula value 3.81). For the Sallen–Key section with $R = 10\,\text{k}\Omega$: $C_2 \approx 11.25$ nF, $C_1 \approx 22.5$ nF give $Q = 0.7071$ exactly. For the ADC, the Butterworth attenuation at 9 kHz with a 1.2 kHz corner is about 70 dB for order 4 and 87.5 dB for order 5; a 12-bit ADC needs about 74 dB, so order 5 is required at $f_s = 10$ kHz.

Aerospace Application Link

A rocket flight computer samples a 16 g accelerometer at 2 kHz to detect motor burnout (a sharp drop in acceleration) within 10 ms. Using your Task 4 group-delay data, choose the filter family and order for the anti-alias filter so that the timing error stays under 2 ms while the alias rejection requirement of Task 6 holds, and quantify the delay of a 5th-order Butterworth versus a 5th-order Bessel filter at a 400 Hz corner.

Deliverables

D1 **First-order analysis:** derivations, response plots, the unbuffered two-stage RC derivation.

D2 **Circuit solver:** the MNA code with tests against analytic transfer functions.

D3 **Filter design:** the Butterworth order calculation, pole plot and the four-family comparison table.

D4 **Sallen–Key and tolerance:** E12 design, Monte Carlo histograms and yield table.

D5 **Anti-alias and measurement:** the ADC filter requirements for both sample rates and the comparison of measured with modelled response.

D6 **Memo (250 words):** "Which anti-alias filter for the burnout detector?" Cite at least two external sources.

Bonus (a) Add a finite op-amp gain–bandwidth product to your solver and show how it degrades a high-Q stage. (b) Design a multiple-feedback band-pass and a state-variable filter. (c) Switched-capacitor filter: derive the equivalent resistor $1/(f_{\text{clk}}C)$.

Learning Resources

Analog Filter Theory and Design

- Sedra, A.S., Smith, K.C. *Microelectronic Circuits*, chapter on filters and tuned amplifiers.
- Schaumann, R., Van Valkenburg, M.E. *Design of Analog Filters*.
- Horowitz, P., Hill, W. *The Art of Electronics*, chapter on active filters.
- Analog Devices, *Op Amp Applications Handbook* and the *Active Filter Design* tutorials (free).

Anti-Aliasing

- Kester, W. (ed.) *Data Conversion Handbook*, Analog Devices (anti-alias filter chapters).
- Zumbahlen, H. (ed.) *Linear Circuit Design Handbook*, Analog Devices (filter chapter).

Tools

- SciPy `scipy.signal` (`buttord`, `butter`, `cheby1`, `bessel`, `ellip`, `freqs`, `group_delay`): https://docs.scipy.org/doc/scipy/reference/signal.html
- ngspice (optional cross-check): https://ngspice.sourceforge.io

Evaluation

Score your own submission out of 100 on five equally weighted criteria, then ask your mentor to score it independently and compare.

Criterion	What to look for	Marks
Conceptual depth	Correct mathematics and physics; every result explained through a mechanism (why the spectrum looks like this), not only through code behaviour.	/20
Implementation quality	Correct, modular, commented, reproducible code; no unexplained magic numbers; analytic checks and self-check targets pass.	/20
Aerospace-application integration	The signal-processing result is carried through to a measurable consequence for a sensor, a link, a flight system or a detection decision, as the task requires.	/20
Analysis and critical thinking	Error bars, convergence, comparison to theory, honest treatment of limitations, false alarms and failure cases.	/20
Communication	Clear report, labelled plots (units, legends, captions) and a memo that a non-expert student could follow.	/20

Bonus tasks are not part of the 100 marks. A bonus attempted badly does not cost you marks, and a well-executed one is the best sign that you own the material.